Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#06)

Use of Venus Simulator for RISC-V Procedure Execution

Student Name: Huzaifa Kashif Roll No: 24I-6501 Section: CE-A

Date performed: 29th September, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad Version: 1.0

Verified by: Dr. Nasir Ali Shah Updated: Fall 2026

Task 1:

Description:

The issue here was the s0 (in procedure:) was being rewritten with 3. Instead i replaced s0 with s1 and other instances of s0 in the procedure block with s1. Another error was that the loop wouldn't end after ecall. I created an end: block at the end of the code. After ecall i used j format instruction to jump to the end block.

Code:

```
.text
main:
li a0, 1
li s0, 2500
jal ra, procedure       # call procedure
add t1, s0, a0          # result should be 1003+2500
li a0,1                 # setup system call for display (command in a0)
mv a1, t1               # data to display in a1
ecall                   # displays result on console
j end
procedure:
li s1, 3
li t0, 1000
add a0, s1, t0
jalr zero, ra, 0 # return
end:
```

Output:



Task 2:

Description:
Here all thats happening is that 5 is being added instantly in register s0, 10 in register t0 and 15 in t1. Then using **addi sp, sp, -12** an offset of 12 is given to stack pointer. 12 offset as we need to store 3 bytes. 12/4=3. Then: **sw s0, 0(sp)** **sw t0, 4(sp)** **sw t1, 8(sp)** Stored each of registers s0,t0 and t1 into the stack. Right afer: **lw t4, 8(sp)** **lw t2, 4(sp)** **lw t3, 0(sp)** This loads that recently stored registers into t4,t2 and t3. Finally: **addi sp, sp, 12** Resets the stack pointer.

Task 3:

Description:
Simply created a eqsolver block. It will jump and link to that block carry out the calculations and save it in a0 where I then initiate the display and print it out.
Code:
.text main: addi a0,x0,10 #a addi a1,x0,5 #b jal ra, eqsolver li a0,1 ecall

```
j end

eqsolver:
add t0,a0,a1
sub t1,a0,a1
mul a1,t0,t1
jalr zero,ra,0

end:
```

Output:



Task 4:

Description:

Similarly, I created two methods areafinder and perimeterfinder and it will jump and link twice and return back twice saving the answer of area in s0 and perimeter in s1.

Code:

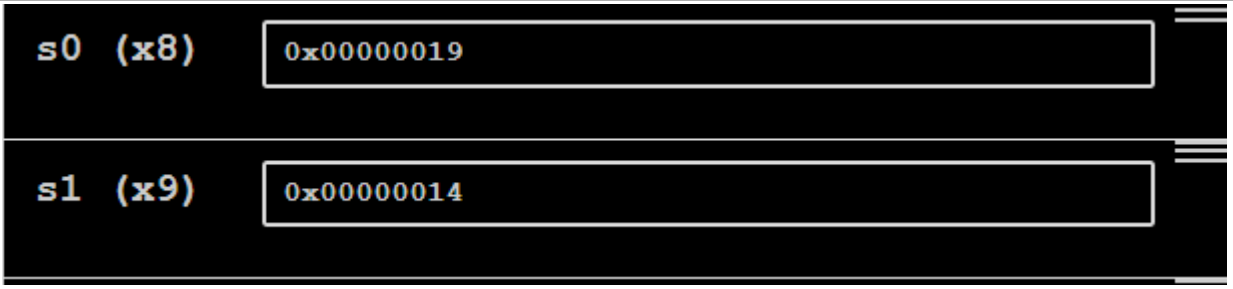
```
.text
main:
addi a1,x0,5 #side
jal ra, areafinder
mv s0, a0
jal ra, perimeterfinder
```

```
mv s1,a0
j end

areafinder:
mul a0,a1,a1
jalr zero,ra,0

perimeterfinder:
addi t0,x0,4
mul a0,a1,t0
jalr zero,ra,0
end:
```

Output:



Task 5:

Description:

Here, I created the eqsolver block and used jump and link to save the return address of the next line. The calculator is done in the eqsolver block and saved in a1. Then it returns back by using jalr to the return address.

Code:

```
.text
main:
addi a1,x0,2 #x
jal ra, eqsolver
li a0,1
ecall
j end

eqsolver:
mul t0,a1,a1
mul t0,t0,a1 # 2^3

addi t1,x0,5

mul t1,t1,a1 #5(2)
mul t1,t1,t1# (5*2)^2

addi t2,x0,6
mul t2,t2,a1 #2*6

add a0,t0,t1
add a1,a0,t2

jalr zero,ra,0
end:
```

Output:



Task 6:

Description:
Here, I created 3 blocks F1,F2,F3. The condition block is where it jump and links from the main. Here all the conditions are checked. Which ever condition is true it will jump to that block. After the calculations each F1,F2 and F3 block contains the instruction jalr to return back to the main block.
Code:

```
.text
main:
addi a0 x0 1 #k
addi a2 x0 5 #n
addi s0 x0 1 #check ==1
jal ra,conditions
li a0,1
ecall
j end

conditions:
beq a0,s0,F1
bge a2,a0,F2
j F3
j end



F1:
mul t0,a2,a2 #x^2
mul t1,a0,a0 #z^2
add a1,t0,t1
jalr zero,ra,0
F2:
mul a1,a2,a2
jalr zero,ra,0
F3:
addi a1,a2,5
jalr zero,ra,0
```

end:

Output:

If k =1 and n= 5



If k =2 and n= 5



If k=6 and n = 5



Task :7

Description:

This task is done in the same way as Task 6. I created 3 blocks flag1,flag2 and flag3. The condition block is where it jump and links from the main. Here all the conditions are checked. Which ever condition is true it will jump to that block. After the calculations each flag1,flag2 and flag3 block contains the instruction jalr to return back to the main block.

Code:

```
.text
main:
addi a0 x0 0 #k
addi a2 x0 5 #n
addi s0 x0 0 #check ==1
jal ra,conditions
li a0,1
ecall
j end

conditions:
beq a0,s0,fun1
beq a2,a0,fun2
j fun3
j end


fun1:
mul a1,a2,a2
jalr zero,ra,0
fun2:
mul t0,a2,a2
mul a1,t0,a2
jalr zero,ra,0
fun3:
addi a1,a2,1
jalr zero,ra,0

end:
```

Output:

k=0 and n=5



k=5 and n=5



k=2 and n=5



Task 8:

Description:
Here, i used a simple loop condition for the non recursion, jumping back into the function if the condtion i<=n is true. For the Recursion, i utilized stack to save f(n-1) values and f(n-2) values. For stack, i gave it and offset of -12 and then restored it by the end.
Code:
Without Recursion: .text main: addi a1,x0,5 #n addi t0,x0,0 #a addi t1,x0,1 #b addi t3,x0,2 #i mv t2,t3 #c fib: beq a1,x0,edgecase forloop: bgt t3,a1, end add t2,t0,t1 mv t0,t1 mv t1,t2 addi t3,t3,1 j forloop

```
edgecase:
addi t1,x0,0
j end
end:
mv a0,t1
```

With Recursion:

```
.text
main:
addi a1,x0,5 #n
addi s0,x0,1 #edgecase 2
jal ra, fib
j end

fib:
beq a1,x0,edgecase1
beq a1,s0,edgecase2

 # Stack frame
addi sp, sp, -12
sw a1, 0(sp)        # n in stack
sw ra, 4(sp)        # ra in stack

   # fib(n-1)
addi a1, a1, -1
jal ra, fib
sw a0, 8(sp)        # fib(n-1)

   # fib(n-2)
lw a1, 0(sp)        # restore n
addi a1, a1, -2
jal ra, fib

lw t0, 8(sp)
add a0, t0, a0

lw ra, 4(sp)
addi sp, sp, 12
jalr zero, ra, 0

edgecase1:
addi a0,x0,0
jalr zero,ra,0
edgecase2:
addi a0,x0,1
jalr zero,ra,0
```

end:

Output:

Without recursion showing b :

Iteration 1:



Iteration 2:



Iteration 3:



Iteration 4:



With Recursion:

